# Practical No: 1

**Aim**: Write a program to Create a class and Implement or demonstrates the concepts of constructor, this and static.

1. Default Constructor:

```
class cons{
    int speed;

    Run | Debug
    public static void main(String[] args){
        cons d = new cons();
        System.out.println(d.speed);
    }
}
```

```
PS D:\Practicals SYCS
AppData\Roaming\Code
0
```

2. Parameterised Constructor:

```
class para {
    String model, model1;
    int price, price1;
    para(){
        this(m: "Tesla", p: 30000000, m1: "Audi", p1: 6000000);
    }
    para(String m, int p, String m1, int p1){
        this.model = m;
        this.price = p;
        this.model1 = m1;
        this.price1 = p1;
    }
    void display(){
        System.out.println("Model: " + model);
        System.out.println("Price " + price);
        System.out.println("Model: " + model1);
        System.out.println("Model: " + price1);
    }
    Run | Debug
    public static void main(String[] args){
        para p = new para();
        p.display();
    }
}
```

```
PS D:\Practicals SYCS
Model: Tesla
Price 30000000
Model: Audi
Model: 6000000
```

3. Copy Constructor:

```java
class car {
    String model;
    car(String m){
        model = m;
    }
    car(car c){
        model = c.model;
    }
    void display(){
        System.out.println("Model: " + model);
    }
    Run | Debug
    public static void main(String[] args){
        car c1 = new car(m: "toyota");
        car c2 = new car(c1);
        c2.display();
    }
}
```

```
PS D:\Practicals SY
Model: toyota
PS D:\Practicals SY
```

4. Using this keyword:

```java
class student {
    String name;
    student(String name){
        this.name = name;
    }

    void show(){
        System.out.println("Name: " + name);
    }

    Run | Debug
    public static void main(String[] args){
        student s = new student(name: "Adnan Shaikh");
        s.show();
    }
}
```

```
PS D:\Practicals SYCS\SYC
Name: Adnan Shaikh
PS D:\Practicals SYCS\SYC
```

5. Static Keyword:

```
class utility {
    static void greet(){
        System.out.println(x: "Hello, Good Evening!");
    }
    Run | Debug
    public static void main(String[] args){
        utility.greet();
    }
}
```

```
PS D:\Practicals SYCS\SYC
Hello, Good Evening!
PS D:\Practicals SYCS\SYC
```

b) **Aim:** Write a program to implement the concept of inheritance and method overriding.

1. Single Inheritance:

```
class animal{
    void makeSound(){
        System.out.println(x: "Animal makes sound a lot!");
    }
}

class dog extends animal{
    void bark(){
        System.out.println(x: "Dogs: Barks on everyting!");
    }
}

public class main{
    Run | Debug
    public static void main(String[] args){
        dog d = new dog();
        d.makeSound();
        d.bark();
    }
}
```

```
AppData\Roaming\Code\User\
Animal makes sound a lot!
Dogs: Barks on everyting!
PS D:\Practicals SYCS\SYCS
```

## 2. Multilevel Inheritance:

```java
class lion {
    void genre1(){
        System.out.println(x: "King of the Jungle");
    }
}
class tiger extends lion{
    void genre2(){
        System.out.println(x: "Wild Predator");
    }
}

class cat extends tiger{
    void genre3(){
        System.out.println(x: "Family friendly");
    }
}

public class main2{
    Run | Debug
    public static void main(String[] args){
        cat c = new cat();
        c.genre1();
        c.genre2();
        c.genre3();
    }
}
```

```
King of the Jungle
Wild Predator
Family friendly
PS D:\Practicals SYCS\
```

## 3. Method Overriding:

```java
class Animal {
    void makeSound(){
        System.out.println(x: "Animal Makes sound a lot!");
    }
}

class Dog extends Animal{
    @Override
    void makeSound(){
        System.out.println(x: "Dog: Barks!");
    }
}

class Cat extends Animal{
    @Override
    void makeSound(){
        System.out.println(x: "Cat: Meow! Meow!");
    }
}
public class zoo{
    Run | Debug
    public static void main(String[] args){
        Animal d = new Dog();
        Animal c = new Cat();
        d.makeSound();
        c.makeSound();
    }
}
```

```
PS D:\Practicals SY
Dog: Barks!
Cat: Meow! Meow!
PS D:\Practicals SY
```

c) **Aim:** Write a program to demonstrate Polymorphism (Using in Method), Abstraction, and Encapsulation:

```
class Calculator {
    int add(int a, int b){
        return a + b;
    }

    double add(double a, double b){
        return a + b;
    }
    int add(int a, int b, int c){
        return a + b + c;
    }
}

public class javac{
    public static void main(String[] args){
        Calculator c = new Calculator();
        System.out.println(c.add(45,64));
        System.out.println(c.add(96.3,58.6));
        System.out.println(c.add(235,254,36));
    }
}
```

```
PS D:\Prac
109
154.9
525
PS D:\Prac
```

d) Abstraction and Encapsulation:

```
class BankAccount {
    private double balance;
    public void deposit(double amount){
        if(amount > 0){
            balance = balance += amount;
        } else {
            System.out.println(x: "Invalid Amount");
        }

    }
    public void withdraw(double amount){
        if(amount > 0 && amount <= balance){
            balance = balance -= amount;
        } else {
            System.out.println(x: "Insufficient funds or Invalid amount");
        }
    }
    public double getBalance(){
        return balance;
    }
}
public class abst{
    Run | Debug
    public static void main(String[] args){
        BankAccount b = new BankAccount();
        b.deposit(amount: 1250);
        b.withdraw(amount: 485);
        System.out.println(b.getBalance());
    }
}
```

```
AppData
765.0
PS D:\P
```

## e) Abstract class and Method:

```java
abstract class vehicle {

    abstract void start();

    void fuel_type(String type){
        System.out.println("The vehicle uses " + type + " Fuel");
    }
}
class three_vehicle extends vehicle{
    @Override
    void start(){
        System.out.println(x: "The Tesla has started");
    }
    void price(int value){
        System.out.println("The price of Tesla is "+ value + " lakhs");
    }
}
class four_vehicle extends three_vehicle{
    @Override
    void start(){
        System.out.println(x: "The audi has started.");
    }
    void price(int value){
        System.out.println("The price of Audi is "+ value + " lakhs");
    }
}

public class motor{
    Run | Debug
    public static void main(String[] args) {
        three_vehicle ob1 = new three_vehicle();
        ob1.start();
        ob1.fuel_type(type: "Petrol");
        ob1.price(value: 3);
        four_vehicle ob2 = new four_vehicle();
        ob2.start();
        ob2.fuel_type(type: "Diesel");
        ob2.price(value: 68);
    }
}
```

```
AppData\Roaming\Code\User\works
The Tesla has started
The vehicle uses Petrol Fuel
The price of Tesla is 3 lakhs
The audi has started.
The vehicle uses Diesel Fuel
The price of Audi is 68 lakhs
PS D:\Practicals SYCS\SYCS\java
```